



第15课时：C++字符串——string

声明 · 输入输出 · 常用操作

string类型的声明与初始化、cin/getline、size/length/[]访问/+拼接



为什么需要字符串？



到目前为止，我们处理的数据主要是**数值**（`int`、`double`）和**单个字符**（`char`）。但在实际编程中，我们经常需要处理**文本**：

- **学生姓名**："张三"
- **一句话**："Hello, World!"
- **网址**："www.luogu.com.cn"
- **一段密码**："Abc123!@#"

字符串就是用来存储和处理文本的数据类型。C++ 提供了强大的 `string` 类型，让文本处理变得简单直观。

字符数组 vs string类型



在C语言中，字符串用**字符数组**表示：

```
char name1[20] = "Hello"; // C风格字符数组
```

C++ 提供了更方便的 **string** 类型：

```
string name2 = "Hello"; // C++ string类型
```

string 类型的优势：

- **自动管理内存**：不需要手动指定大小
- **丰富的内置函数**：`size()`、`substr()`、`find()` 等
- **支持运算符**：`+` 拼接、`==` 比较、`[]` 访问
- **更安全**：不容易越界出错

💡 从现在起，我们统一使用 **string** 类型来处理字符串。



string的声明与初始化



使用 `string` 类型需要包含 `<string>` 头文件（`<bits/stdc++.h>` 已包含）。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    // 方式1: 默认构造（空字符串）
    string s1;

    // 方式2: 用字符串字面量初始化
    string s2 = "Hello";

    // 方式3: 用另一个string初始化
    string s3 = s2;

    // 方式4: 用n个相同字符初始化
    string s4(5, 'A');           // "AAAAA"

    // 方式5: 用C风格字符数组初始化
    char arr[] = "World";
    string s5(arr);             // "World"

    cout << s1 << "|" << s2 << " " << s3 << " "
         << s4 << " " << s5 << endl;
    // 输出: |Hello Hello AAAAA World
    return 0;
}
```

string的输入——cin



`cin >>` 是最常用的字符串输入方式。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string name;
    cout << "请输入你的名字: ";
    cin >> name;
    cout << "你好, " << name << "! " << endl;
    return 0;
}
// 输入: Alice
// 输出: 你好, Alice!
```

⚠ **cin 的局限：**遇到空格、Tab、回车就会停止读取。

```
输入: Hello World
name 只会得到 "Hello", "World" 留在输入缓冲区
```



string的输入——getline



如果字符串中**包含空格**，需要使用 `getline()` 函数。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string sentence;
    cout << "请输入一句话: ";
    getline(cin, sentence);
    cout << "你输入的是: " << sentence << endl;
    return 0;
}
// 输入: Hello World C++
// 输出: 你输入的是: Hello World C++
```

`getline(cin, s)` 会读取一整行，直到遇到回车才结束，**包括空格**。



cin与getline混合使用的陷阱



当 `cin >>` 和 `getline()` 混合使用时，可能出现问题：

```
int n;  
string name;  
cin >> n;           // 输入数字后按回车  
getline(cin, name); // 此处会读取到空字符串！
```

原因： `cin >> n` 读取数字后，回车符 `\n` 留在了缓冲区，`getline` 会立即读到它并结束。

解决方法： 在 `cin >>` 和 `getline()` 之间加一个 `cin.ignore()`：

```
cin >> n;  
cin.ignore();           // 清除缓冲区中的回车符  
getline(cin, name);    // 正确读取
```



string的输出



使用 `cout <<` 即可输出字符串。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello, World!";
    cout << s << endl;           // 输出整个字符串

    // 可以与其他内容混合输出
    int n = 42;
    cout << "字符串: " << s << ", 数字: " << n << endl;
    // 输出: 字符串: Hello, World!, 数字: 42

    // 也可以用 printf 输出 (需用 .c_str())
    printf("%s\n", s.c_str());
    // 输出: Hello, World!
    return 0;
}
```

获取字符串长度——size()与length()



`size()` 和 `length()` 功能完全相同，返回字符串中的**字符个数**。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello";
    cout << "size() = " << s.size() << endl;    // 5
    cout << "length() = " << s.length() << endl; // 5

    string s2 = "你好C++";
    cout << "s2.length() = " << s2.length() << endl;
    // 注意：中文字符占多个字节，结果可能不是4
    return 0;
}
```

 **提示：** `size()` 和 `length()` 返回的是 `size_t` 类型（无符号整数），比较时注意类型匹配，避免 `s.length() - 1` 在空字符串时产生意外。

用下标访问字符——[]运算符



和数组一样，可以用 `[]` 访问字符串中的**单个字符**，下标从 **0** 开始。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello";
    cout << s[0] << endl;    // 输出 H (第1个字符)
    cout << s[1] << endl;    // 输出 e (第2个字符)
    cout << s[4] << endl;    // 输出 o (第5个字符)

    // 修改字符
    s[0] = 'h';
    cout << s << endl;      // 输出 hello

    // 最后一个字符
    cout << s[s.size() - 1] << endl; // 输出 o
    return 0;
}
```

⚠ 注意：下标范围是 `0` ~ `s.size() - 1`，越界访问会导致未定义行为。

字符串拼接——+运算符



`string` 类型支持用 `+` 运算符进行拼接，非常直观。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s1 = "Hello";
    string s2 = " ";
    string s3 = "World";

    // 用 + 拼接
    string result = s1 + s2 + s3;
    cout << result << endl;    // 输出: Hello World

    // 用 += 追加
    string s4 = "C++";
    s4 += " is";
    s4 += " fun!";
    cout << s4 << endl;        // 输出: C++ is fun!

    // 也可以拼接字符
    string s5 = "AB";
    s5 += 'C';
    cout << s5 << endl;        // 输出: ABC
    return 0;
}
```

💡 注意： `+` 两侧至少有一个必须是 `string` 类型，不能直接 `"a" + "b"`。



string的其他常用操作



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello";

    // at() 访问: 与 [] 类似, 但会做越界检查
    cout << s.at(0) << endl;    // H

    // back() 和 front()
    cout << s.front() << endl;  // H (第一个字符)
    cout << s.back() << endl;   // o (最后一个字符)

    // empty() 判断是否为空
    cout << s.empty() << endl;  // 0 (false, 非空)

    // clear() 清空字符串
    s.clear();
    cout << s.empty() << endl;  // 1 (true, 已空)

    // push_back() 追加一个字符
    s.push_back('A');
    s.push_back('B');
    cout << s << endl;          // AB
    return 0;
}
```

综合例题1：字符串信息卡



题目：输入一个字符串，输出其长度、首字符、末字符，并将所有字符改为大写后输出。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;

    // 输出长度
    cout << "长度: " << s.size() << endl;
    // 输出首末字符
    cout << "首字符: " << s[0] << endl;
    cout << "末字符: " << s[s.size() - 1] << endl;

    // 转大写 (仅小写字母)
    for (int i = 0; i < s.size(); i++) {
        if (s[i] >= 'a' && s[i] <= 'z') {
            s[i] = s[i] - 'a' + 'A';
        }
    }
    cout << "大写: " << s << endl;
    return 0;
}
// 输入: hello
// 输出: 长度: 5 / 首字符: h / 末字符: o / 大写: HELLO
```

综合例题2：字符串拼接与拼接



题目：输入姓和名（各一个单词），拼接成全名并输出，再输出全名长度。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string firstName, lastName;
    cout << "请输入姓：";
    cin >> firstName;
    cout << "请输入名：";
    cin >> lastName;

    // 拼接全名
    string fullName = lastName + " " + firstName;
    cout << "全名：" << fullName << endl;
    cout << "名字长度：" << fullName.size() << endl;
    // 注意：空格也算一个字符
    return 0;
}
// 输入：张 / 三
// 输出：全名：三 张 / 名字长度：3
```

cin vs getline 对比



特性	<code>cin >></code>	<code>getline(cin, s)</code>
分隔符	空格、Tab、回车	仅回车
能否读取空格	不能	能
适用场景	单个单词	整行文本
缓冲区残留	分隔符留在缓冲区	回车被消费
混合使用注意	后接getline需ignore	一般无残留

选择建议：输入**单个单词**用 `cin >>`，输入**含空格的整行**用 `getline()`。

常见错误与调试



错误类型	示例/说明	解决方法
cin后getline读空行	<code>cin >> n;</code> 后 <code>getline</code> 读到空字符串	中间加 <code>cin.ignore()</code>
[]越界访问	<code>s[s.size()]</code> 访问越界位置	有效下标为 <code>0</code> ~ <code>s.size()-1</code>
"a" + "b" 编译错误	两个C风格字符串字面量不能直接相加	至少一边用 <code>string</code> 变量
忘加头文件	使用 <code>string</code> 但未包含 <code><string></code>	用 <code>#include <bits/stdc++.h></code>
size()返回无符号数	<code>s.size() - 1</code> 在空串时下溢	先判断 <code>s.empty()</code> 或用 <code>int</code> 接收

调试技巧： 在输入后立即输出字符串内容确认是否正确读取；用断点或输出检查下标值。



知识小结



- ✓ **string声明与初始化:** `string s = "Hello";` , 支持多种初始化方式
- ✓ **输入输出:** `cin >>` 读单个单词, `getline(cin, s)` 读整行, `cout <<` 输出
- ✓ **cin与getline混用:** 中间加 `cin.ignore()` 清除缓冲区回车
- ✓ **size() / length():** 返回字符串长度 (字符个数)
- ✓ **[]访问:** 下标从0开始, 可读可写, 范围 `0` ~ `size()-1`
- ✓ **+拼接:** `string + string` 或 `string += "文字"` , 直观方便
- ✓ **其他操作:** `at()`、`front()`、`back()`、`empty()`、`clear()`、`push_back()`



下节课预告



第16课时：字符串常用操作

- 字符串比较
- 子串查找与提取
- 字符与ASCII码
- 字符串的遍历与修改

"字符串是程序与人沟通的桥梁，掌握它，你就掌握了文本世界的钥匙。"

谢谢大家!



string让C++的文本处理变得优雅而强大!

Q & A